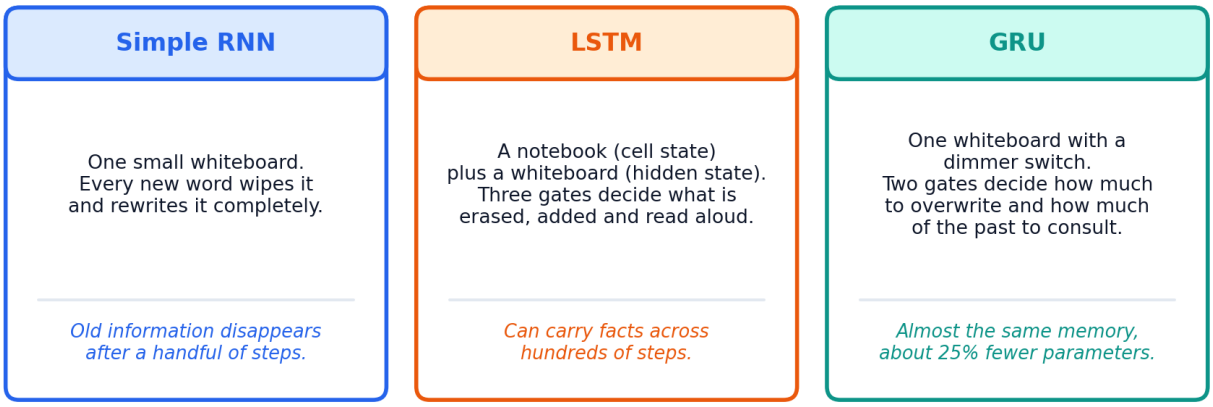
A VISUAL GUIDE FOR BEGINNERS

Recurrent Neural Networks, LSTM and GRU

Understanding how neural networks learn from sequences — built up slowly, from the very first idea to working code.

Three ways of remembering



What you need before starting	What you will be able to do at the end
Basic algebra and vectors	Explain why plain networks fail on sequences
What a neural network and a weight are	Read and interpret the RNN, LSTM and GRU equations
The idea of training with gradient descent	Describe what every gate does and why it exists
A little Python (optional, for the code sections)	Choose the right model and build one in Keras or PyTorch

Contents

Part 1 Getting oriented

1 Why sequences need a different kind of network 3

Part 2 The simple recurrent network

2 How an RNN actually works 5

3 Training an RNN, and where it breaks 9

Part 3 Long Short-Term Memory

4 The LSTM: memory with gates 12

Part 4 The Gated Recurrent Unit

5 The GRU: same idea, fewer parts 18

Part 5 Using them in practice

6 RNN vs LSTM vs GRU: choosing one 22

7 A practical build guide with code 24

8 Where recurrent networks stand today 28

Appendix

A Glossary of every term used 29

B One-page formula cheat sheet 31

C Practice questions with answers 32

CHAPTER 1

Why sequences need a different kind of network

Almost everything interesting in the world arrives as a **sequence** — a list of things where the *order matters*. The words of a sentence, the beats of a heart on an ECG, the daily closing price of a share, the frames of a video, the notes of a song. If you shuffle them, the meaning is destroyed.

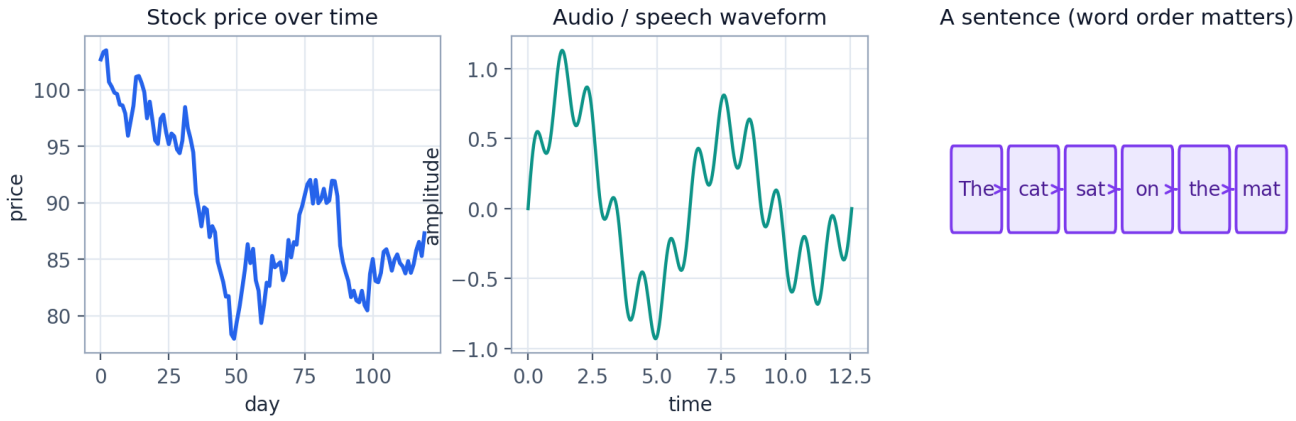


Figure 1.1 — Three ordinary examples of sequential data. In each case, what comes next depends on what came before.

1.1 Two properties that make sequences hard

Ordinary neural networks (the feedforward kind you meet first) are built on two quiet assumptions that sequences immediately violate.

- **Fixed length.** A feedforward network has a fixed number of input slots. But sentences have 3 words or 300 words, and you do not know in advance which.
- **Independent examples.** A feedforward network treats each input as unrelated to the last one. But in *“I grew up in France… I speak fluent ____”*, the correct answer depends on a word that appeared much earlier.

You could try to patch this. You could fix a window of, say, five words and slide it along. That is exactly what an *n*-gram model does, and it works — until the useful clue sits six words back. You could also pad every sentence to 500 slots, but then the network has to learn the meaning of the word *“cat”* separately in position 1, position 2, position 3 and so on, because each slot has its own weights. That is enormously wasteful.

> **KEY IDEA**
> We want a network that (a) accepts sequences of **any length**, (b) uses the **same weights** at every position, and (c) carries information **forward through time**. A recurrent neural network does all three with one small trick: it keeps a memory and feeds it back into itself.

1.2 The trick: a loop

A recurrent neural network (RNN) is a normal network with one extra wire. The hidden layer's output is copied and fed back in as an extra input on the next step. That copied value is called the **hidden state**, written h_t, and it is the network's memory of everything it has seen so far.

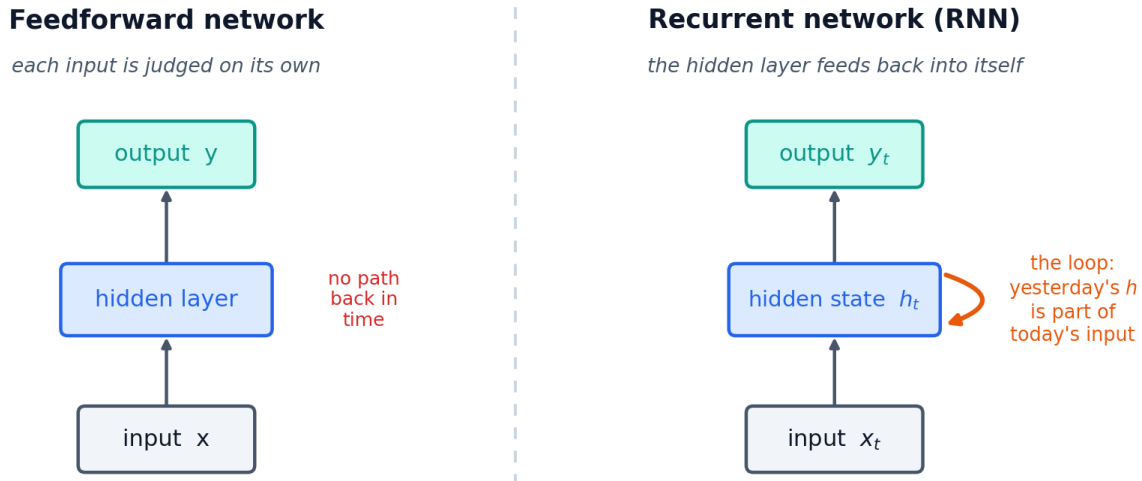


Figure 1.2 — The only structural difference between a feedforward network and an RNN is the orange feedback wire. Everything else — weights, activations, gradient descent — is unchanged.

> **IN PLAIN ENGLISH**
> Think of reading a novel. You do not re-read the first 200 pages every time you start a new sentence. You carry a compressed summary in your head, and you update that summary as you read. The hidden state h_t is that running summary; the recurrent weights are the rule for updating it.

1.3 The three characters in this story

The rest of this guide follows three models, each solving a problem left over by the one before.

Model	Year	The problem it solves	Cost
Simple RNN	1986	Gives a network a memory at all, and handles variable-length input.	Its memory fades after roughly 5–10 steps.
LSTM	1997	Adds a protected memory lane plus three gates, so information can survive hundreds of steps.	Four times as many parameters; slower.
GRU	2014	Achieves almost the same thing with two gates and one state instead of three gates and two states.	Slightly less flexible on very hard tasks.

Read them in order. Each one only makes sense as an answer to the previous one's weakness.

CHAPTER 2

How an RNN actually works

We will now open up the loop and look at exactly what happens at each step. This chapter is the foundation for everything after it: an LSTM and a GRU are the same machine with a smarter update rule.

2.1 Unrolling: the picture that makes it click

The loop drawing is compact but confusing, because it hides time. The standard cure is to **unroll** the network: draw one copy of the cell for each time step and connect them in a chain.

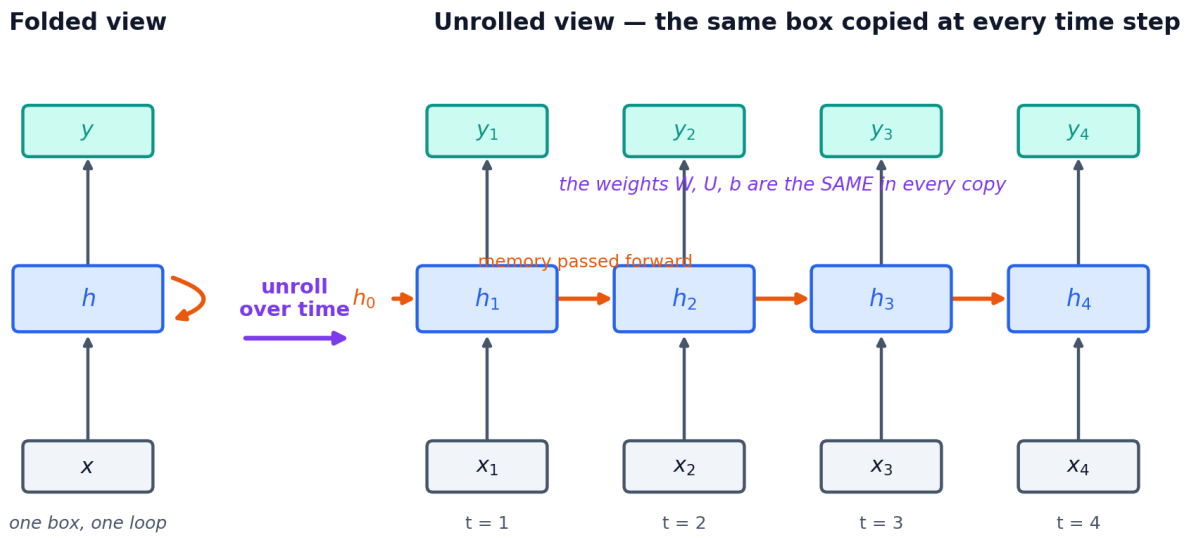


Figure 2.1 — The same RNN, folded (left) and unrolled over four time steps (right). Unrolled, an RNN is simply a very deep feedforward network in which every layer shares one set of weights.

KEY IDEA

The copies are **not** separate layers with separate parameters. They are the *same* cell reused. A 100-word sentence unrolls into 100 copies, and all 100 share one matrix W, one matrix U and one bias b. This is why an RNN can read a sentence of any length — you simply unroll it further.

2.2 The equations, one symbol at a time

An RNN cell does two things at each step: update the memory, then (optionally) produce an output.

$$h_t = \tanh\left(W_{xh}\, x_t \;+\; W_{hh}\, h_{t-1} \;+\; b_h\right)$$

Step 1 — the memory update. This is the heart of the RNN.

$$y_t = W_{hy}\, h_t + b_y$$

Step 2 — the output (add softmax on top for classification).

Every symbol has a job:

Symbol	Name	What it is	Typical shape
x_t	input at time t	The current item — one word vector, one sensor reading, one price.	(input_size,)
h_{t-1}	previous hidden state	The summary of everything seen before now. At t = 1 it is usually all zeros.	(hidden_size,)
W_{xh}	input weights	How strongly each part of the new input affects the memory. **Learned.**	(hidden, input)
W_{hh}	recurrent weights	How the old memory carries into the new memory. **Learned.** This is the recurrence.	(hidden, hidden)
b_h	bias	A learned offset.	(hidden,)
tanh	activation	Squashes the result into the range −1 to +1 so the memory cannot blow up.	—
h_t	new hidden state	The updated summary, passed to the next step *and* used to make this step's prediction.	(hidden,)

IN PLAIN ENGLISH

Read the update equation out loud as a sentence: *"My new memory is a squashed blend of what I just saw and what I already remembered."* That is genuinely all a simple RNN does.

2.3 Why tanh, and why it matters later

The activation is not decoration. Without it, stacking the update 50 times would just multiply the input by a matrix 50 times, and the values would race off to infinity or collapse to zero. tanh keeps every component of h politely between −1 and +1.

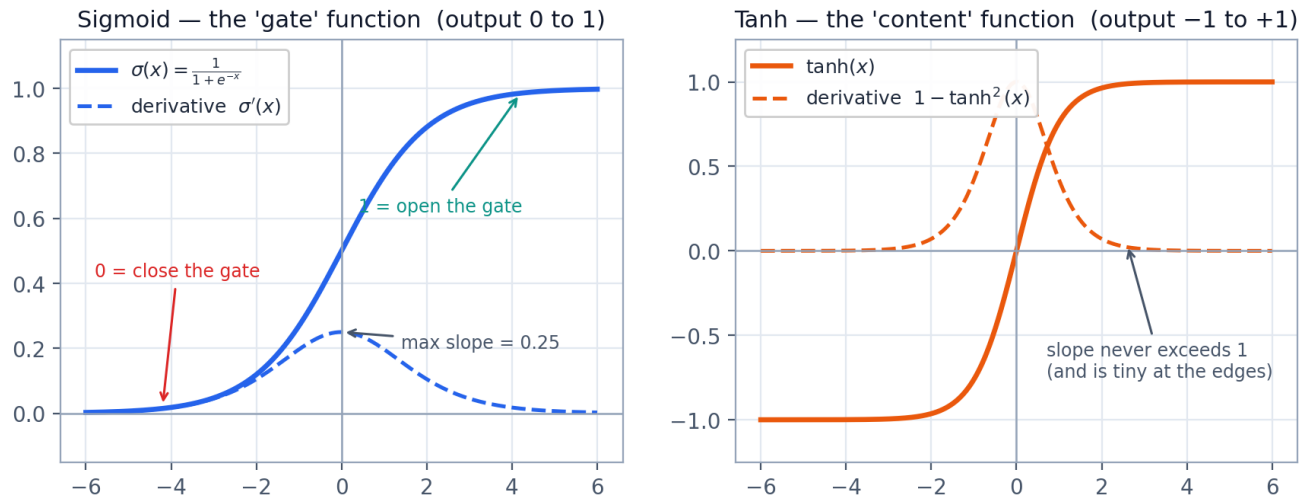


*Figure 2.2 — The two functions that appear everywhere in this guide. **Sigmoid** outputs 0 to 1, which is why it is used for gates (Chapter 4). **tanh** outputs −1 to +1, which is why it is used for content. Note the dashed derivative curves — they are the reason RNNs are hard to train.*

Look carefully at the dashed lines. The slope of tanh is at most 1, and it is nearly 0 whenever the input is large. The slope of sigmoid is at most 0.25. During training we multiply many of these slopes together, once per time step. Multiplying numbers smaller than one, over and over, drives

the product towards zero. Hold that thought until Chapter 3 — it is the single most important fact in this guide.

2.4 A fully worked example with real numbers

Nothing removes mystery faster than doing the arithmetic. We will use the smallest possible RNN: one input number, one hidden number. The three learned parameters have been fixed at arbitrary values.

$$W_{xh} = 0.5, \quad W_{hh} = 0.8, \quad b_h = 0.1, \quad h_0 = 0$$

Our (pretend) trained parameters and the starting memory.

We feed in the sequence **x = [1.0, 0.5, −1.0, 2.0]** and apply $h_t = \tanh(0.5 \cdot x_t + 0.8 \cdot h_{t-1} + 0.1)$ four times.

t	input x_t	old memory h_{t-1}	$0.5 \cdot x_t$	$0.8 \cdot h_{t-1}$	sum + b	h_t = tanh(sum)
1	1.00	0.0000	+0.500	+0.0000	+0.6000	**+0.5370**
2	0.50	0.5370	+0.250	+0.4296	+0.7796	**+0.6525**
3	-1.00	0.6525	-0.500	+0.5220	+0.1220	**+0.1214**
4	2.00	0.1214	+1.000	+0.0971	+1.1971	**+0.8328**

Three things are worth noticing in that table. First, the memory at step 4 still carries a trace of step 1 — the value from h_1 was multiplied by 0.8, then 0.8 again, and so on. Second, that trace is shrinking: 0.8 × 0.8 × 0.8 = 0.512, so by step 4 only about half of step 1's influence survives, and after 20 steps only 1.2% would remain. Third, the arithmetic is identical at every step — that is weight sharing in action.

WATCH OUT

Change W_{hh} from 0.8 to 1.3 and re-run the table. The internal sums grow every step until tanh saturates at +1 and stops responding to input at all. Too small and the memory dies; too large and it jams. A simple RNN has an uncomfortably narrow safe zone, and nothing in its design keeps it there.

2.5 Real RNNs work with vectors, not single numbers

In practice x_t and h_t are vectors and the weights are matrices, but nothing conceptually changes. A common convention glues the two inputs together into one vector and uses a single matrix:

$$h_t = \tanh\left(W \cdot [\, h_{t-1},\ x_t \,] + b\right)$$

The square brackets mean concatenation. This compact form is the one used in the LSTM and GRU diagrams later.

If the input has 100 dimensions and the hidden state has 128, then W is a 128 × 228 matrix and b has 128 entries — 29,312 learned numbers in total, reused at every single time step. Remember this number; we will compare it with the LSTM and GRU in Chapter 6.

2.6 Five ways to wire an RNN to a task

The same cell can be arranged to solve very different problems, depending on where you attach inputs and read outputs.

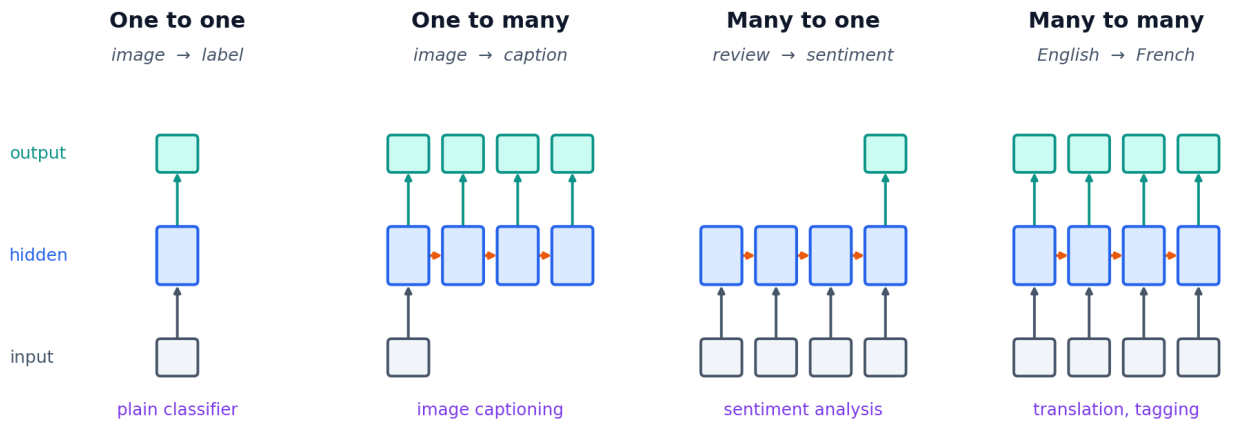


Figure 2.3 — Four common input/output arrangements. Grey = input, blue = the recurrent hidden state, green = output. A fifth arrangement, the encoder–decoder, chains a many-to-one onto a one-to-many and is the basis of machine translation.

Arrangement	Input → output	Real example
One to one	single → single	Ordinary image classification (no recurrence needed — shown only for completeness)
One to many	single → sequence	Image captioning; generating music from a starting seed
Many to one	sequence → single	Sentiment of a review; is this ECG trace abnormal; next-day price forecast
Many to many (aligned)	sequence → sequence, same length	Part-of-speech tagging; naming entities in a sentence; frame-by-frame video labelling
Encoder–decoder	sequence → sequence, different length	Translation; summarisation; question answering

CHAPTER 3

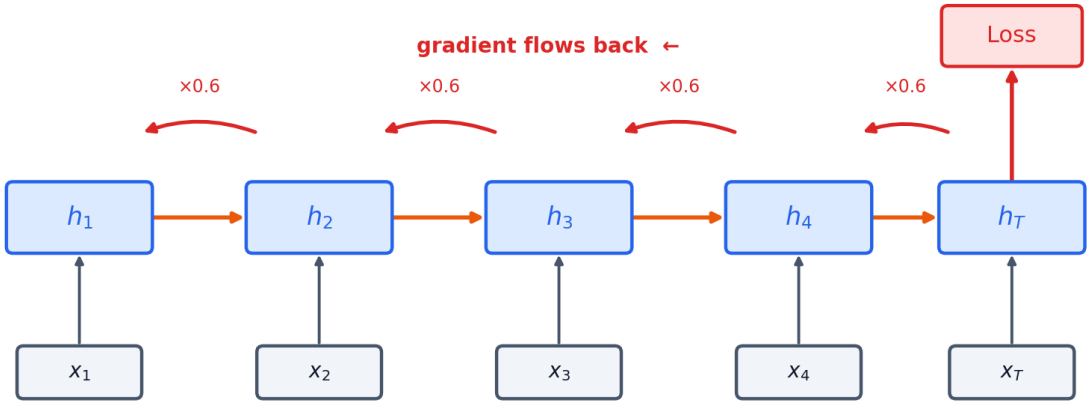
Training an RNN, and where it breaks

An RNN learns the same way any network does: make a prediction, measure the error, work out how each weight contributed to that error, nudge every weight in the direction that reduces it. The only twist is that the weights were used many times, at many time steps, so their contributions must be gathered from all of them.

3.1 Backpropagation Through Time (BPTT)

Unroll the network, run the whole sequence forwards, compute the loss, then propagate the error backwards through every step. Because W_{hh} was used at every step, its final gradient is the **sum** of the gradients computed at each step.

Backpropagation Through Time: the error travels backwards through every step



each backward hop multiplies the gradient again — after many hops it is either microscopic (vanishing) or enormous (exploding)

Figure 3.1 — BPTT. The forward pass (orange) builds the memory left to right; the backward pass (red) carries the error right to left, and gets multiplied by a factor at every hop.

1. **Forward.** Feed x_1 … x_T through the shared cell, storing every h_t along the way.
2. **Loss.** Compare the prediction with the target — cross-entropy for classification, mean squared error for regression.
3. **Backward.** Send the error back through the chain, one step at a time.
4. **Accumulate.** Add up each weight's gradient contribution from all T steps.
5. **Update.** Apply one optimiser step (Adam, SGD) using the summed gradients.

PRACTICAL TIP

Full BPTT over a 10,000-step sequence is impossibly slow and memory-hungry. Real systems use **truncated BPTT**: carry the hidden state forward indefinitely, but only backpropagate through the last *k* steps (commonly 30–200). You lose some very-long-range learning signal and gain a training run that actually finishes.

3.2 The vanishing and exploding gradient problem

Here is the flaw that defines this whole field. To send the error from step t back to step k, the chain rule forces you to multiply one factor for every step in between:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} W_{hh}^{\top} \cdot \operatorname{diag}\left(1 - \tanh^2(z_j)\right)$$

One factor per time step. The same matrix, multiplied by itself, again and again.

Multiplying the same quantity by itself many times has only three possible outcomes, and two of them are disasters.

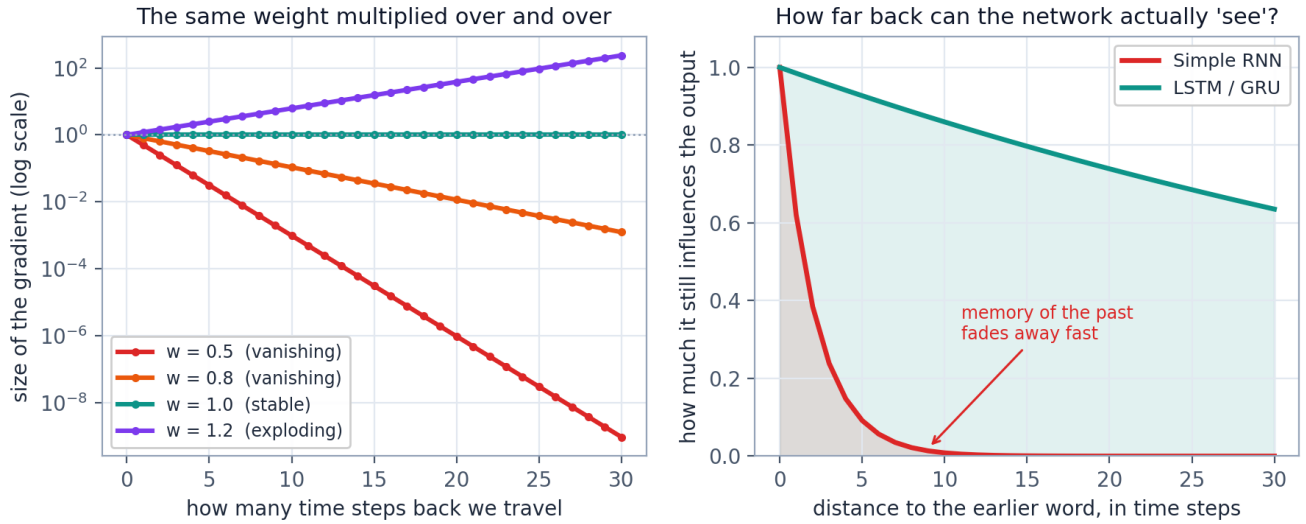


Figure 3.2 — Left: what repeated multiplication does to a gradient. Right: the practical consequence — how far into the past a model can still be influenced by. Only the knife-edge value 1.0 is stable, and training never lands exactly there.

If the repeated factor is…	The gradient…	What you see in practice
< 1 (vanishing)	shrinks towards zero exponentially	Loss drops a little then stalls. The model learns local patterns but never links distant ones. **This is the common case**, because tanh's slope is at most 1 and usually well below it.
> 1 (exploding)	grows without bound	Loss suddenly becomes NaN or leaps to a huge value; weights are destroyed in a single update. Dramatic, but easy to detect and easy to fix.
= 1	stays constant	Ideal, and essentially impossible to maintain by luck.

WATCH OUT

The 0.9 example. Suppose each backward hop multiplies the gradient by 0.9 — a mild, entirely realistic amount of shrinkage. After 10 steps the signal is at 35%. After 50 steps it is 0.5%. After 100 steps it is 0.003%. The network is not *refusing* to learn long-range dependencies; it is receiving almost no gradient telling it to. This is why a simple RNN cannot connect *"France"* at word 3 to *"French"* at word 60.

3.3 The partial fixes

Several repairs help, and you should know them, but only the last one truly solves the problem.

Fix	How it works	Verdict
Gradient clipping	If the gradient's norm exceeds a threshold (say 5), scale it back down before the update.	**Solves exploding completely.** Cheap, standard, always use it. Does nothing for vanishing.
Careful initialisation	Start W_{hh} as an orthogonal or identity matrix so its repeated product stays near 1 at the beginning.	Helps early training. The benefit erodes as weights move.
ReLU instead of tanh	ReLU's slope is exactly 1 for positive inputs, so it does not shrink the gradient.	Trades vanishing for exploding; needs careful tuning.
Skip / residual connections	Let information hop over time steps directly.	Genuinely useful; the same principle the LSTM uses.
Gated architectures	Redesign the cell so there is a path along which the gradient is *added* rather than repeatedly multiplied.	**The real solution.** This is the LSTM and the GRU, and it is where we go next.

KEY IDEA

The idea that unlocks everything in Chapter 4: if repeated **multiplication** is what kills the gradient, build a route through the network where the memory is updated by **addition** instead. Addition has a derivative of exactly 1, so a gradient travelling along that route is not shrunk at all.

CHAPTER 4

The LSTM: memory with gates

The Long Short-Term Memory network was published by Sepp Hochreiter and Jürgen Schmidhuber in 1997, and for roughly twenty years it was the default tool for anything sequential. The name is worth parsing: it is not “long-short” memory, it is a **short-term memory that lasts a long time** — a working memory that does not evaporate after five steps.

4.1 The two changes

An LSTM makes exactly two changes to the simple RNN. Everything else — unrolling, weight sharing, BPTT — is identical.

1. **A second state, the cell state c_t.** It runs straight through the cell with almost nothing done to it: no matrix multiplication, no activation function. Just one multiplication and one addition. This is the protected memory lane.
2. **Three gates.** Small learned sub-networks that decide, at each step and for each component of the memory, what to erase, what to write, and what to reveal.

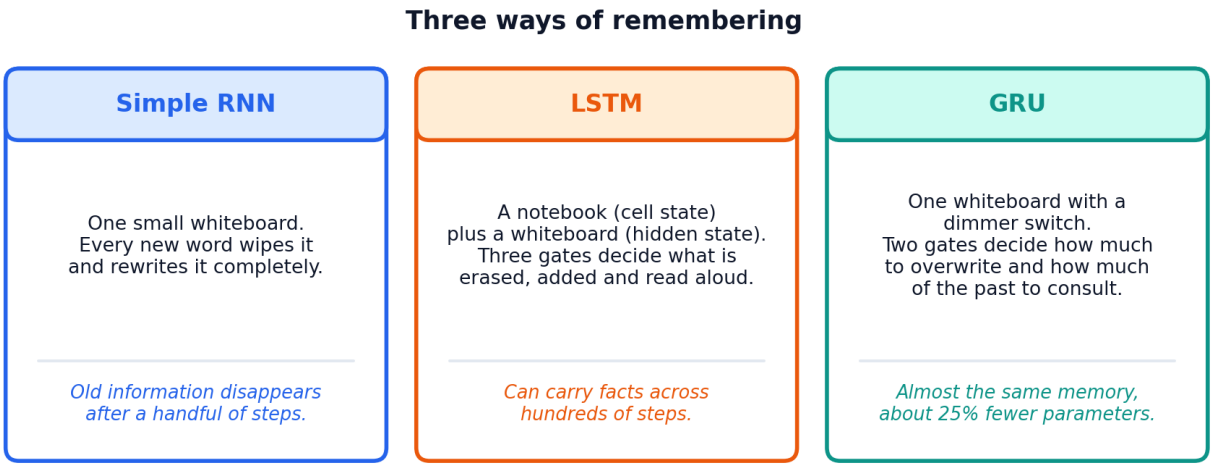


Figure 4.1 — The intuition in one picture. The simple RNN has one surface that gets overwritten; the LSTM adds a durable notebook it can edit selectively.

4.2 What a gate actually is

The word “gate” sounds elaborate. It is not. A gate is a vector of numbers between 0 and 1, produced by a tiny sigmoid layer, which gets multiplied element by element with the thing being gated.

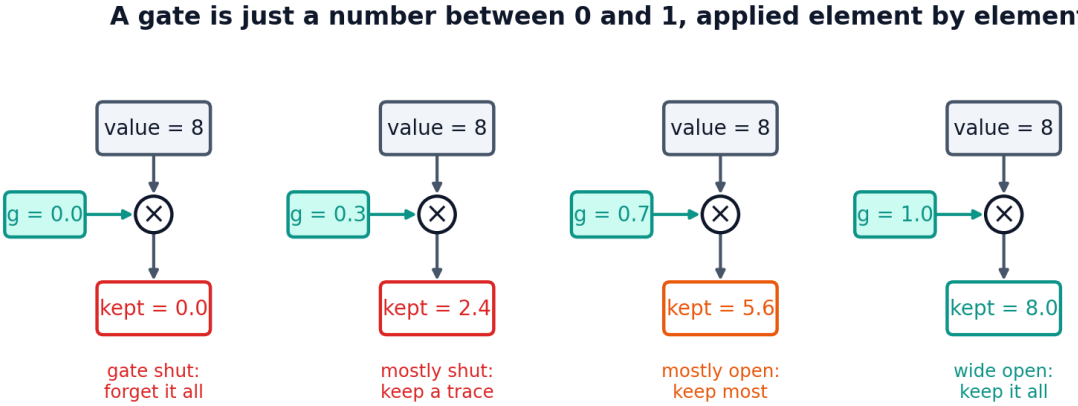


Figure 4.2 — A gate applied to a single value. 0 erases, 1 keeps, and everything in between partially attenuates. Real gates do this independently for every one of the hundreds of components of the memory vector.

> **IN PLAIN ENGLISH**
> A gate is a set of dimmer switches, one per memory slot, and the network learns how to set them based on what it is currently reading. Crucially, the gate value is *computed fresh at every time step* from h_{t-1} and x_t — it is a decision, not a fixed parameter. The same LSTM can hold a fact for 200 steps in one sentence and discard it after 2 steps in the next.

The symbol ⊙ (or × in our diagrams) means **element-wise multiplication**, also called the Hadamard product: multiply the first entry by the first entry, the second by the second, and so on. It is not a matrix product.

4.3 The complete LSTM cell

Here is the whole thing. Take a minute with it — then we will walk each path separately.

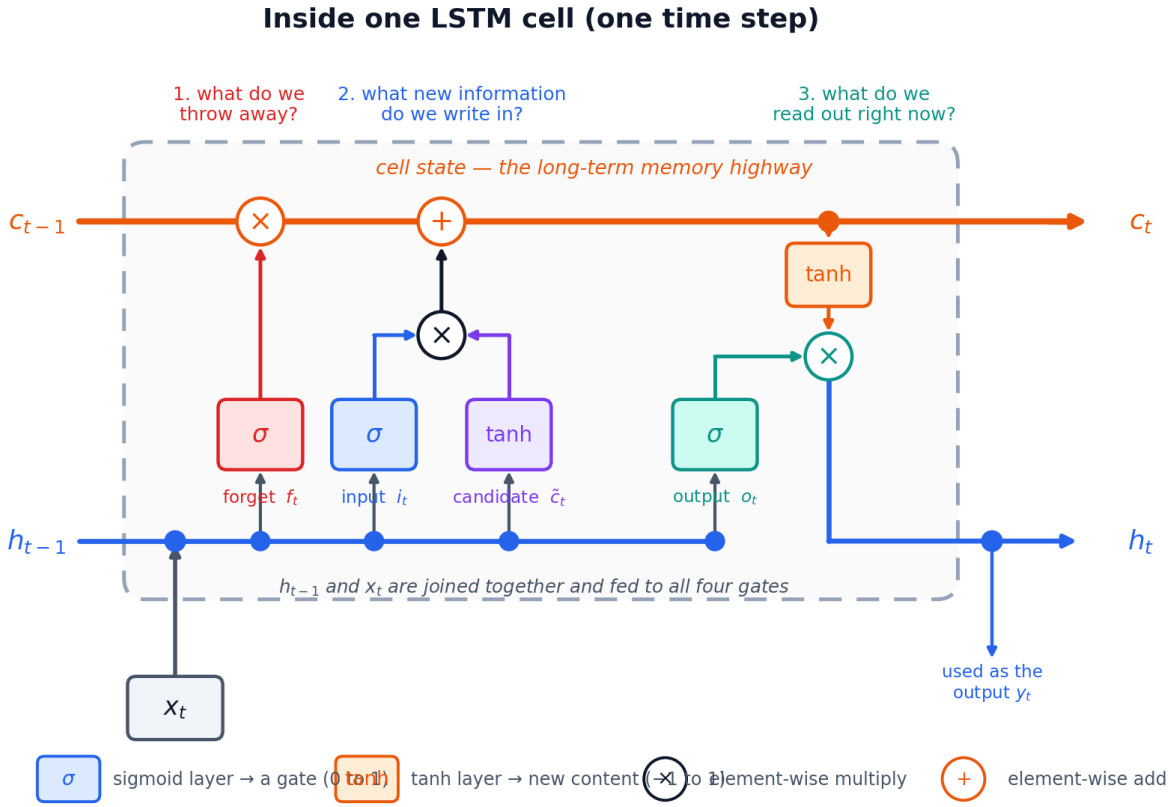


Figure 4.3 — One LSTM cell. The orange lane along the top is the cell state; notice that it only ever meets a multiply and an add. The blue lane along the bottom carries the hidden state. All four gate layers read the same joined vector $[h_{t-1}, x_t]$.

The cell state's journey is the thing to fix in your mind. It enters on the left as c_{t-1}, gets multiplied by the forget gate, gets a new contribution added, and leaves on the right as c_t. No weight matrix ever touches it directly. That is precisely what lets a gradient travel back along it without decaying.

4.4 Step 1 — the forget gate: what should we erase?

$$f_t = \sigma(W_f \cdot [\, h_{t-1},\, x_t \,] + b_f)$$

The forget gate looks at what we remembered and what we are reading now, and outputs a number between 0 and 1 for *every slot* of the cell state. That vector is then multiplied into the old memory:

$$\text{(applied as)} \quad f_t \odot c_{t-1}$$

IN PLAIN ENGLISH

Example. A language model is tracking the gender of the current subject so it can choose *he* or *she* later. When a new subject appears — *"...Sarah left. **David** then said..."* — the forget gate drops close to 0 on the gender slots, wiping the stale fact, while staying near 1 on unrelated slots such as tense or topic.

4.5 Step 2 — the input gate: what should we write?

Writing new information is deliberately split into two questions: *what* could be written, and *how much of it* should actually be written.

$$i_t = \sigma(W_i \cdot [\, h_{t-1},\, x_t \,] + b_i)$$

The input gate — how much to write (0 to 1).

$$\tilde{c}_t = \tanh\left(W_c \cdot [\, h_{t-1},\, x_t \,] + b_c\right)$$

The candidate — what could be written (−1 to +1).

Splitting them matters. The candidate is free to propose a strong change in either direction, and the gate independently decides whether this is a moment worth recording. A determiner like *“the”* usually gets a near-zero input gate; a proper noun usually gets a wide-open one.

4.6 Step 3 — update the cell state

Now combine the two decisions. This single line is the most important equation in the guide.

$$c_t \;=\; f_t \odot c_{t-1} \;+\; i_t \odot \tilde{c}_t$$

Erase some of the old, add some of the new.

KEY IDEA

Why the LSTM defeats vanishing gradients. Differentiate the line above with respect to c_{t-1} and you get f_t — not a weight matrix, not a tanh slope, just the forget gate. If the network learns to keep f_t near 1 for a slot it cares about, the gradient flows back through 100 steps multiplied by roughly 1×1×…×1, and arrives intact. The additive update creates a gradient highway. This is the whole reason the architecture exists.

PRACTICAL TIP

Because of this, the forget gate's bias b_f is often initialised to **+1** rather than 0. That starts the gate wide open, so the cell remembers by default and has to learn to forget — a much easier starting point than the reverse. Keras does this automatically with unit_forget_bias=True.

4.7 Step 4 — the output gate: what should we reveal?

The cell state may hold a great deal that is not relevant to the current prediction. The output gate filters it down to a working hidden state.

$$o_t = \sigma(W_o \cdot [\, h_{t-1},\, x_t \,] + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

h_t goes to the next time step and is also this step's output.

IN PLAIN ENGLISH

The cell state is the network's private notebook; the hidden state is what it says out loud. A model may be quietly tracking the subject, the tense and the topic all at once in c, but when the next word is clearly an adjective it reveals only the slots relevant to adjectives.

4.8 All six equations together

$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	forget gate — what to erase
$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	input gate — how much to write
$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$	candidate — what could be written
$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	output gate — what to reveal
$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	the new cell state
$h_t = o_t \odot \tanh(c_t)$	the new hidden state

The first four lines are four copies of the same small network with different weights. The last two lines are the only real machinery.

4.9 Watching the gates work

It helps to see plausible gate values on a real sentence. The sentence below has a long-range dependency: the adjective at the end describes the noun near the beginning, eight words earlier.

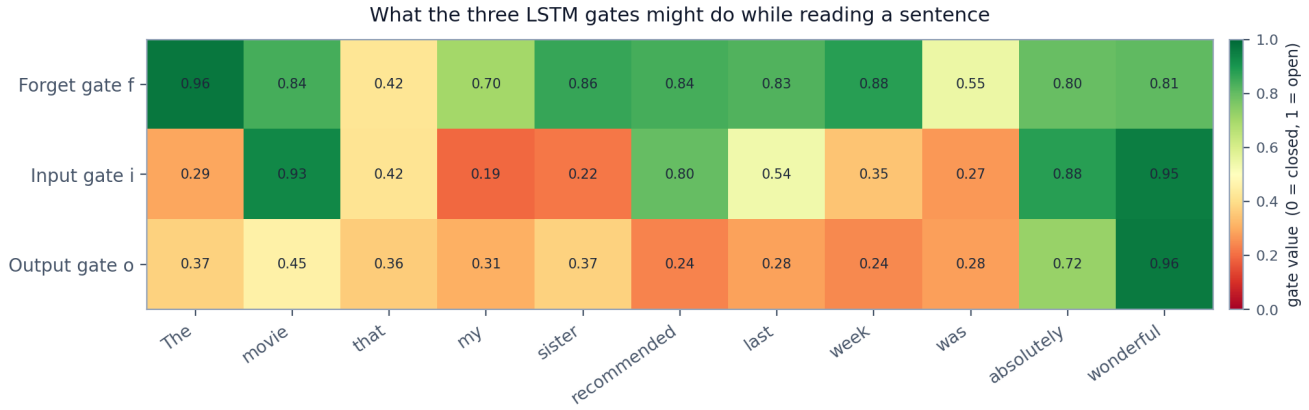


Figure 4.4 — Illustrative gate values while reading a sentence. Green means open (near 1), red means closed (near 0). The forget gate stays high — keep remembering movie — and dips at clause boundaries. The input gate spikes on content words and stays low on filler. The output gate only opens wide when a prediction is actually due.

4.10 A worked example with real numbers

As with the RNN, we shrink everything to one dimension. The weights below are invented; what matters is the pattern of the arithmetic.

$$f_t = \sigma(1.0\,x_t + 0.5\,h_{t-1} - 0.5)$$

$$i_t = \sigma(0.8\,x_t + 0.4\,h_{t-1} - 0.2)$$

$$\tilde{c}_t = \tanh(1.2\,x_t + 0.6\,h_{t-1})$$

$$o_t = \sigma(0.9\,x_t + 0.3\,h_{t-1} - 0.1)$$

Starting from $c_0 = 0$ and $h_0 = 0$, with the input sequence $x = [1.0, 0.0, 2.0]$.

t	x_t	f_t	i_t	candidate $\tilde{c}_t$	o_t	$c_t = f{\cdot}c_{t-1} + i{\cdot}\tilde{c}_t$	$h_t = o{\cdot}\tanh(c_t)$
1	1.0	0.622	0.646	+0.834	0.690	**+0.5383**	**+0.3392**
2	0.0	0.418	0.484	+0.201	0.500	**+0.3222**	**+0.1559**
3	2.0	0.829	0.812	+0.986	0.852	**+1.0680**	**+0.6716**

At step 2 the input is 0, so the input gate almost closes (0.451) and very little new information is written — yet the forget gate is 0.399, so the memory from step 1 is largely retained rather than wiped. The cell is doing something a simple RNN structurally cannot: *choosing* to hold a value steady across a step that carries no information.

4.11 Variants you will meet

Variant	What changes	When you would use it
Peephole LSTM	The gates are also allowed to look at c_{t-1} directly, not only at h_{t-1}.	Precise timing tasks. Rarely used today.
Coupled forget/input	Sets $i_t = 1 - f_t$: only write where you just erased.	Fewer parameters. This idea leads directly to the GRU.
Bidirectional LSTM	Runs one LSTM forwards and a second one backwards, then joins the two hidden states.	Whenever the whole sequence is available in advance — tagging, classification. **Not** for live forecasting.
Stacked / deep LSTM	The hidden states of layer 1 become the inputs of layer 2, and so on.	2–3 layers usually help on large datasets; beyond that the returns fade quickly.

CHAPTER 5

The GRU: same idea, fewer parts

The Gated Recurrent Unit was introduced by Kyunghyun Cho and colleagues in 2014, seventeen years after the LSTM, as part of work on machine translation. The motivation was straightforward: the LSTM works, but does it really need three gates and two separate states? The GRU's answer is no — two gates and one state get you almost all of the benefit for about 25% less computation.

5.1 The two simplifications

1. **Merge the two states.** Drop the separate cell state. The GRU keeps one vector, h_t, which serves as both long-term memory and working output.

2. **Merge forget and input into one gate.** Instead of independently deciding how much to erase and how much to write, a single **update gate** z_t does both: whatever fraction it writes, it erases exactly that much. A second **reset gate** r_t controls how much of the past the new candidate is even allowed to see.

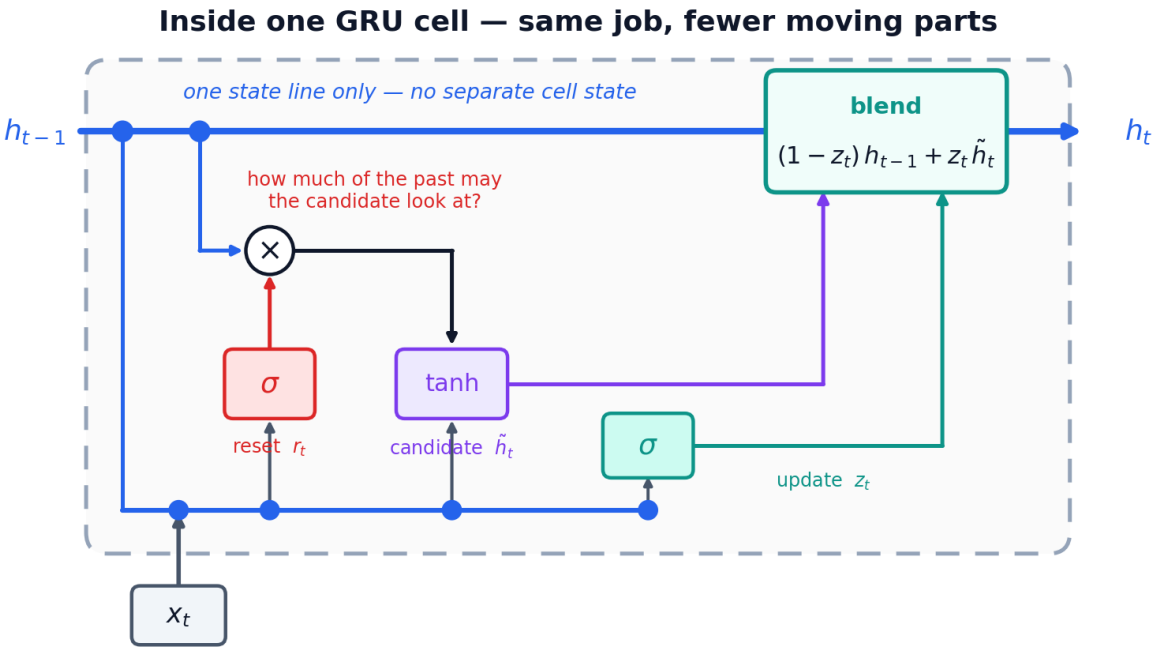


Figure 5.1 — One GRU cell. Compare it with Figure 4.3: one state line instead of two, three learned layers instead of four, and the erase/write decision handled by a single blend at the end.

5.2 The equations

$$r_t = \sigma(W_r \cdot [\, h_{t-1},\, x_t \,] + b_r)$$

Reset gate *— how much of the old state may influence the new candidate. Near 0 means "ignore the past, start fresh".*

$$z_t = \sigma(W_z \cdot [\, h_{t-1},\, x_t \,] + b_z)$$

Update gate *— how much of the state to replace this step. Near 0 means “keep what I had”.*

$$\tilde{h}_t = \tanh\left(W_h \cdot [\, r_t \odot h_{t-1},\, x_t \,] + b_h\right)$$

Candidate state *— note that the reset gate is applied to h_{t-1} before it enters here.*

$$h_t \;=\; (1 - z_t) \odot h_{t-1} \;+\; z_t \odot \tilde{h}_t$$

The blend. *A weighted average of the old state and the candidate, with the weights chosen per slot by z_t.*

KEY IDEA

The final line is a **linear interpolation** — a slider between “keep the past” and “take the new value”. Because the two coefficients are forced to add up to 1, the state cannot grow without bound, and the (1 − z) term plays exactly the role the forget gate played in the LSTM: when z is near 0, h passes through untouched and the gradient with it.

5.3 Understanding the two gates separately

	Reset gate r_t	Update gate z_t
Question it answers	How much of the past should the *new proposal* take into account?	How much of the state should actually *change* this step?
Where it acts	Inside the candidate, before the tanh	At the very end, on the blend
r or z near **0**	The candidate ignores h_{t-1} and is built from x_t alone — useful at the start of a new sentence or clause.	Almost nothing changes; the old state is carried forward. **This is how the GRU remembers across long gaps.**
r or z near **1**	The candidate sees the full past, behaving like a plain RNN update.	The state is replaced by the fresh candidate; the past is dropped.
LSTM counterpart	Roughly the job the output gate does — controlling what the next computation gets to see.	The forget and input gates fused into one dial.

IN PLAIN ENGLISH

One sentence for each gate. **Reset:** *“How much history is relevant to what I am about to propose?”* **Update:** *“How much should I actually change my mind?”*

5.4 A worked example with real numbers

Same treatment as before: one dimension, invented weights, honest arithmetic.

$$r_t = \sigma(0.7\,x_t + 0.4\,h_{t-1} + 0.1)$$

$$z_t = \sigma(0.9\,x_t + 0.5\,h_{t-1} - 0.3)$$

$$\tilde{h}_t = \tanh(1.1\,x_t + 0.8\,(r_t\,h_{t-1}))$$

$$h_t = (1 - z_t)\,h_{t-1} + z_t\,\tilde{h}_t$$

Starting from $h_0 = 0$, with the input sequence $x = [1.0, 0.0, 2.0]$.

t	x_t	r_t	z_t	$r_t \cdot h_{t-1}$	candidate $\tilde{h}_t$	$1 - z_t$	h_t (blend)
1	1.0	0.690	0.646	+0.0000	+0.8005	0.354	**+0.5168**
2	0.0	0.576	0.490	+0.2978	+0.2338	0.510	**+0.3783**
3	2.0	0.839	0.844	+0.3174	+0.9853	0.156	**+0.8907**

Step 2 is the interesting one again. With x = 0 the update gate falls to 0.436, so 56% of the previous state survives untouched into the new state. No matrix multiplication stands between h_1 and h_2 along that path — which is exactly the property that lets gradients travel backwards without dying.

PRACTICAL TIP

Empirically, LSTMs and GRUs perform within a percentage point or so of each other on most benchmarks, with neither winning consistently. The honest advice is: **try the GRU first** because it trains faster, and switch to an LSTM if your sequences are very long or you have plenty of data and the GRU has plateaued.

5.5 LSTM and GRU side by side

Every practical difference between the two architectures, in one place:

	LSTM	GRU
States carried forward	Two: cell state c_t and hidden state h_t	One: hidden state h_t
Gates	Three: forget, input, output	Two: reset, update
Learned layers per step	Four (f, i, candidate, o)	Three (r, z, candidate)
Parameters (hidden n, input m)	$4 \times (n(m+n) + n)$	$3 \times (n(m+n) + n)$
Erase and write decisions	Independent — can erase without writing, or write without erasing	Coupled — whatever is written displaces exactly that much
Exposes full memory?	No — the output gate hides part of c_t	Yes — the whole state is exposed every step
Speed per epoch	Baseline	Typically 20–30% faster
Data efficiency	Needs a little more data to justify its extra parameters	Often better on small and medium datasets
Best known for	Very long dependencies; the safe default on large corpora	Speed and simplicity; strong on short and medium sequences

CHAPTER 6

RNN vs LSTM vs GRU: choosing one

You now know all three architectures. This chapter is about picking one and knowing what it will cost you.

6.1 Counting the parameters

Every gate is one small dense layer over the joined vector $[h_{t-1}, x_t]$, which has length $m + n$ where m is the input size and n the hidden size. One such layer costs $n(m + n)$ weights plus n biases, so:

$$\text{params} = k \cdot (n\,(m+n) + n) \qquad k = 1 \text{ (RNN)},\ \ 3 \text{ (GRU)},\ \ 4 \text{ (LSTM)}$$

With an input of 128 and a hidden size of 128, that is 32,896 parameters for a simple RNN, 98,688 for a GRU and 131,584 for an LSTM — for a single layer, reused at every time step.

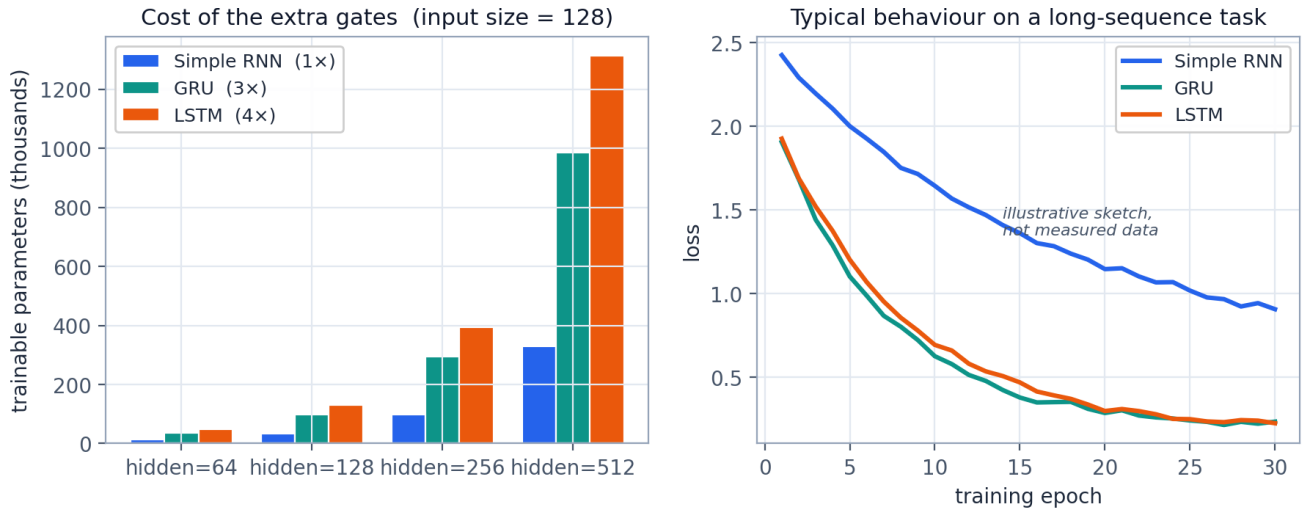


Figure 6.1 — Left: parameter count grows with the number of gates and quadratically with hidden size. Right: the behaviour you typically observe on a task with long-range dependencies — the gated models reach a much lower loss, and the simple RNN plateaus early.

6.2 A decision guide

Your situation	Start with	Why
Sequences under ~10 steps	**Simple RNN**	Vanishing gradients have no time to bite. Fewer parameters, faster, less to overfit.
Sequences of 10–100 steps, modest data	**GRU**	The sweet spot. Handles the range easily and its smaller parameter count is an advantage when data is limited.
Sequences over ~100 steps	**LSTM**	The separate cell state and independent gates give it more room to preserve information over long spans.
Training time or deployment size is the binding constraint	**GRU**	Roughly 25% fewer parameters and 20–30% faster per epoch, at very similar accuracy.
Whole sequence available in advance (tagging, classification)	**Bidirectional GRU or LSTM**	Reading in both directions almost always beats reading in one, when you are allowed to.
Live forecasting, streaming, real-time control	**Unidirectional GRU or LSTM**	A bidirectional model would need to see the future. It cannot be used here.
Long text, translation, summarisation, large dataset	**Transformer**	See Chapter 8 — recurrence is no longer the right tool at this scale.

6.3 A fair mental benchmark

Task	Simple RNN	GRU	LSTM
Remember an input 5 steps back	Good	Good	Good
Remember an input 50 steps back	Fails	Good	Good
Remember an input 500 steps back	Fails	Struggles	Struggles
Training speed	Fastest	Fast	Slowest
Parameters at n = m = 128	32.9k	98.7k	131.6k
Risk of overfitting small data	Low	Medium	Higher

WATCH OUT

Note the third row. Gates **reduce** gradient decay dramatically; they do not abolish it. Dependencies spanning many hundreds of steps remain genuinely hard for any recurrent model, which is a large part of why attention-based models took over.

CHAPTER 7

A practical build guide with code

Everything up to here has been conceptual. This chapter is the part you will come back to when you actually build something.

7.1 Getting the data into the right shape

Recurrent layers in every major framework expect a 3-dimensional tensor:

$$(\text{batch size},\ \text{time steps},\ \text{features})$$

Getting there usually takes four steps.

1. **Tokenise or window.** Text becomes a list of integer token ids. A time series becomes overlapping windows — for example, use days 1–30 to predict day 31, then days 2–31 to predict day 32.
2. **Pad or truncate.** Sequences in one batch must be the same length. Pad short ones with zeros and pass a mask so the model ignores the padding. Choose the length from your data — often the 90th or 95th percentile rather than the maximum.
3. **Embed (text only).** An Embedding layer maps each token id to a dense vector. Never feed raw integer ids into a recurrent layer; the model would read token 500 as five times token 100.
4. **Normalise (numeric only).** Scale features to roughly zero mean and unit variance. Fit the scaler on the training split only, never on the whole dataset, or you leak information from the future into the past.

> **WATCH OUT**
> **The most common beginner error in time-series work** is shuffling before splitting. If you shuffle and then split, some training windows sit after some test windows in real time, the model effectively sees the answers, and your validation score becomes meaningless. Always split *chronologically*: train on the earliest portion, validate on the middle, test on the most recent.

7.2 Keras

A two-layer bidirectional model for classifying text. To try a different cell type, change one word:

Keras — sentiment classification (many-to-one)

```
import tensorflow as tf
from tensorflow.keras import layers, models

VOCAB, MAXLEN, EMB, HID = 20000, 200, 128, 128

model = models.Sequential([
    layers.Input(shape=(MAXLEN,)),
    layers.Embedding(VOCAB, EMB, mask_zero=True),   # mask_zero handles padding

    # swap in SimpleRNN or GRU here - the interface is identical
    layers.Bidirectional(layers.LSTM(HID, return_sequences=True, dropout=0.2)),
    layers.Bidirectional(layers.LSTM(64)),          # last layer: no return_sequences

    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid'),
])

model.compile(optimizer=tf.keras.optimizers.Adam(1e-3, clipnorm=1.0),
              loss='binary_crossentropy', metrics=['accuracy'])

model.fit(x_train, y_train, epochs=20, batch_size=64,
          validation_data=(x_val, y_val),
          callbacks=[tf.keras.callbacks.EarlyStopping(patience=3,
                                      restore_best_weights=True)])
```

Two details are easy to miss. return_sequences=True makes a layer output the hidden state at *every* step, which is what the next recurrent layer needs; the final recurrent layer leaves it off so that only the last state is passed to the dense head. And clipnorm=1.0 is gradient clipping from Section 3.3 — it costs nothing and prevents exploding gradients.

7.3 PyTorch

The same architecture written as a module, with the training step spelled out:

PyTorch — the same model

```
import torch, torch.nn as nn

class SeqClassifier(nn.Module):
    def __init__(self, vocab, emb=128, hid=128, layers_n=2, classes=2):
        super().__init__()
        self.emb  = nn.Embedding(vocab, emb, padding_idx=0)
        # nn.RNN / nn.GRU / nn.LSTM all share this signature
        self.rnn  = nn.LSTM(emb, hid, num_layers=layers_n, batch_first=True,
                            bidirectional=True, dropout=0.2)
        self.drop = nn.Dropout(0.5)
        self.fc   = nn.Linear(hid * 2, classes)    # *2 because bidirectional

    def forward(self, x):
        e = self.emb(x)                       # (batch, time, emb)
        out, (h_n, c_n) = self.rnn(e)         # GRU returns only h_n
        last = torch.cat([h_n[-2], h_n[-1]], dim=1)   # join both directions
        return self.fc(self.drop(last))

model = SeqClassifier(vocab=20000)
opt   = torch.optim.Adam(model.parameters(), lr=1e-3)
lossf = nn.CrossEntropyLoss()

for xb, yb in train_loader:
    opt.zero_grad()
    loss = lossf(model(xb), yb)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)   # gradient clipping
    opt.step()
```

PRACTICAL TIP

Remember batch_first=True. PyTorch's recurrent layers default to the shape (time, batch, features), which is the opposite of what almost everyone expects and a reliable source of silent bugs.

7.4 Hyperparameters worth knowing

Setting	Sensible starting point	Notes
Hidden size	64–256	The main capacity dial. Double it if the model underfits; halve it if train and validation loss diverge.
Number of layers	1–2	A second layer often helps. A third rarely does unless the dataset is very large.
Dropout	0.2 between recurrent layers, 0.5 before the output	Frameworks apply the same dropout mask at every time step, which is the correct (variational) behaviour for recurrent nets.
Learning rate	1e-3 with Adam	If the loss oscillates, drop to 3e-4. Reducing the rate on plateau is a reliable free win.
Gradient clipping	norm 1.0 or 5.0	Always on. Cheap insurance against exploding gradients.
Batch size	32–128	Larger batches train faster but generalise slightly worse on small datasets.
Sequence length	90th percentile of your data	Long tails waste computation on padding for very little gain.
Epochs	As many as needed, with early stopping	Patience of 3–5 epochs on validation loss, restoring the best weights.

7.5 Symptoms and cures

What you see	Likely cause	What to do
Loss becomes NaN	Exploding gradients, or a learning rate that is far too high	Turn on gradient clipping; lower the learning rate; check for NaNs in the input data.
Loss drops fast then flattens well above zero	Vanishing gradients, or too little capacity	Switch from SimpleRNN to GRU or LSTM; increase hidden size; add a layer.
Training loss falls, validation loss rises	Overfitting	More dropout; smaller hidden size; more data or augmentation; stop earlier.
Both losses stay flat from the start	Learning rate too low, or the input is not informative	Raise the learning rate by 10×; verify the data pipeline by overfitting a single batch on purpose.
Excellent validation score, poor real-world performance	Data leakage — almost always a shuffled time-series split	Re-split chronologically; refit scalers on the training portion only.
Training is extremely slow	Long sequences, or a custom loop that cannot use the fused kernels	Truncate sequences; use the built-in layers; use a GRU; sort batches by length to reduce padding.

PRACTICAL TIP

The single best debugging habit: before training on the full dataset, take one batch of about eight examples and train on it until the model fits it almost perfectly. If it cannot memorise eight examples, there is a bug in your model or data pipeline, and no amount of tuning will save you. This takes two minutes and saves entire afternoons.

CHAPTER 8

Where recurrent networks stand today

It would be dishonest to end without saying this plainly: for most large-scale language tasks, recurrent networks have been replaced by **Transformers**. Understanding why is a good test of whether Chapters 1 to 5 have landed.

8.1 The two problems recurrence could not solve

- **It is inherently sequential.** To compute h_{100} you must first compute h_{99}, which needs h_{98}. The whole chain resists parallelisation, which is exactly the wrong property for GPU hardware. A Transformer processes every position at once.
- **Long-range information is still squeezed through one vector.** Everything the model knows about the first 400 words must fit into a single fixed-size h. Attention removes the bottleneck entirely by letting any position look directly at any other, in one step rather than 400.

8.2 Where recurrent models are still the right answer

- **Short sequences.** Under roughly 50 steps, a GRU is often as accurate as a Transformer, and much cheaper.
- **Small datasets.** Transformers are data-hungry. With a few thousand examples, a GRU's built-in assumption that order matters locally is a genuine advantage.
- **Streaming and low-latency inference.** An RNN's state update is constant cost per step, no matter how long the history. Standard attention costs grow with the length of the context.
- **Embedded and edge devices.** A small GRU running on a microcontroller for keyword spotting or sensor classification remains completely standard practice.
- **Classic time series.** Sensor data, demand forecasting and anomaly detection are still very often handled well by an LSTM or GRU.

KEY IDEA

The concepts do not become obsolete even where the architecture does. Gating reappears in modern models as gated linear units and gated residuals; the additive-update trick behind the LSTM cell state is the same idea as the residual connections that make deep Transformers trainable. And state-space models such as Mamba are, in a real sense, recurrence brought back with a design that parallelises. Learning LSTMs is not learning history.

APPENDIX A

Glossary of every term used

Term	Meaning
Activation function	A non-linear function applied to a layer's output. tanh and sigmoid are the two that matter here.
Backpropagation Through Time	Backpropagation applied to an unrolled RNN. Gradients for the shared weights are summed across all time steps.
Bidirectional	Two recurrent layers, one reading forwards and one backwards, with their outputs joined. Requires the full sequence up front.
Candidate state	The proposed new content ($\tilde{c}_t$ in an LSTM, $\tilde{h}_t$ in a GRU) before a gate decides how much of it to accept.
Cell state	The LSTM's protected long-term memory lane, c_t. Modified only by one element-wise multiply and one add.
Element-wise (Hadamard) product	Written ⊙. Multiply matching entries of two equal-length vectors. Not a matrix product.
Embedding	A learned dense vector representing a discrete token; the standard first layer for text.
Exploding gradient	Gradients growing without bound during backpropagation, producing NaN losses. Cured by gradient clipping.
Forget gate	The LSTM gate that decides which parts of the cell state to erase.
Gate	A vector of values in [0, 1] produced by a sigmoid layer and multiplied element-wise into another vector, controlling how much passes through.
Gradient clipping	Rescaling a gradient whose norm exceeds a threshold, before the optimiser step.
GRU	Gated Recurrent Unit (2014). One state, two gates — reset and update.
Hidden state	The running summary h_t carried from one time step to the next; in an RNN and GRU it is also the layer's output.
Input gate	The LSTM gate that decides how much of the candidate to write into the cell state.
LSTM	Long Short-Term Memory (1997). Two states, three gates.
Masking	Telling the model which time steps are padding so it can ignore them.
Output gate	The LSTM gate that decides how much of the cell state is revealed as the hidden state.
Padding	Filling short sequences with a dummy value so all sequences in a batch have equal length.
Reset gate	The GRU gate controlling how much of the previous state the candidate is allowed to see.
Sequence length / time steps	How many items the model reads before producing its final answer.

Term	Meaning
Truncated BPTT	Backpropagating through only the most recent k steps while still carrying the hidden state forward, to make training tractable.
Unrolling	Drawing or building one copy of the recurrent cell per time step. The copies share one set of weights.
Update gate	The GRU gate that decides how much of the state to replace with the candidate.
Vanishing gradient	Gradients shrinking towards zero as they travel back through time, preventing the model from learning long-range dependencies.
Weight sharing	Using the same parameters at every time step. It is what lets an RNN handle sequences of any length.

APPENDIX B

One-page formula cheat sheet

Simple RNN

$$h_t = \tanh\left(W \cdot [h_{t-1}, x_t] + b\right) \qquad y_t = W_y h_t + b_y$$

LSTM — three gates, two states

$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	forget: what to erase
$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	input: how much to write
$\tilde{c}_t = \tanh\left(W_c \cdot [h_{t-1}, x_t] + b_c\right)$	candidate: what to write
$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	output: what to reveal
$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$	new cell state
$h_t = o_t \odot \tanh(c_t)$	new hidden state

GRU — two gates, one state

$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$	reset: how much past the candidate sees
$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$	update: how much to change
$\tilde{h}_t = \tanh\left(W_h \cdot [r_t \odot h_{t-1},\ x_t] + b_h\right)$	candidate state
$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	blend old and new

Parameter count for one layer

$$k \cdot (n(m+n) + n), \qquad k = 1,\ 3,\ 4 \ \text{for RNN, GRU, LSTM}$$

m = input size, n = hidden size.

The three sentences to remember

1. An RNN is a network with a memory that it rewrites completely at every step.

2. An LSTM adds a protected memory lane updated by *addition*, plus three gates that decide what to erase, write and reveal — which is why its gradients survive.

3. A GRU does the same job with one state and two gates by coupling the erase and write decisions into a single blend.

APPENDIX C

Practice questions

Try each one before reading the answer. If you can answer all twelve you have understood this guide.

Q1. Why can a feedforward network not handle a sentence of unknown length?

A. Its input layer has a fixed number of slots, and each slot has its own weights. An RNN reuses one set of weights at every position, so it simply unrolls further for a longer sentence.

Q2. What exactly is shared between the unrolled copies of an RNN cell?

A. All the parameters — W_{xh}, W_{hh} and b. The copies differ only in the input they receive and the hidden state they inherit.

Q3. Why does the vanishing gradient problem exist at all?

A. Sending a gradient back k steps requires multiplying k factors together. If each is below 1, the product shrinks exponentially towards zero.

Q4. Which fixes exploding gradients, and does it also fix vanishing gradients?

A. Gradient clipping fixes exploding gradients completely. It does nothing for vanishing gradients, because a gradient that is already near zero cannot be rescued by rescaling.

Q5. Why is the update $c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$ so important?

A. Its derivative with respect to c_{t-1} is just f_t. With f near 1 the gradient passes back through many steps essentially unchanged — addition, not repeated matrix multiplication.

Q6. Why do gates use sigmoid rather than tanh?

A. A gate is a proportion, so it must lie in [0, 1]. Sigmoid gives exactly that; tanh spans −1 to +1 and is used instead for *content*, where a negative value is meaningful.

Q7. What is the difference between the cell state and the hidden state?

A. The cell state is the private long-term memory, modified only by a multiply and an add. The hidden state is the filtered public view of it, produced by the output gate, and it is what the next layer and the next time step actually see.

Q8. Why is the forget-gate bias often initialised to +1?

A. It starts the gate near 1, so the cell remembers by default and must learn to forget. Starting near 0 would erase the memory before the network has learned anything.

Q9. How does the GRU cover the job of the LSTM's forget and input gates with one gate?

A. It couples them: $h_t = (1-z) \odot h_{t-1} + z \odot \tilde{h}_t$. Whatever fraction z writes, exactly the fraction 1−z of the old state is kept, so erasing and writing are two sides of one dial.

Q10. What does the GRU's reset gate do that the update gate does not?

A. It controls how much of the previous state the *candidate* is computed from. Near 0 it lets the model propose a fresh state built almost entirely from the current input — useful at a clause or sentence boundary.

Q11. An LSTM with input size 64 and hidden size 100 — how many parameters?

A. $4 \times (100 \times (64 + 100) + 100) = 4 \times 16{,}500 =$ **66,000**. A GRU would use 49,500 and a simple RNN 16,500.

Q12. **Your validation accuracy on a stock-price model is 97%, but it loses money live. What is the first thing to check?**

A. Data leakage from a shuffled split. Time-series data must be split chronologically, and any scaler must be fitted on the training portion only.

NEXT STEPS

Where to go next. Build a character-level text generator with a two-layer LSTM — it is small enough to train on a laptop and it makes every idea in this guide concrete. After that, read about sequence-to-sequence models with attention, which is the bridge from these architectures to modern Transformers.